

WA3872 Lab Guide

Introduction to Command Line, Git, and IDEs for Leaders



Part of
Accenture

© 2026 Ascendient, LLC

Revision 2.1.0 published on 2026-05-04

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

🚨 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Evergreen Insurance Quote Page — First Look.....	5
1.1. A Note About Keyboard Shortcuts.....	6
1.2. Why You're Doing This Lab.....	6
1.3. What's Already Set Up on Your Lab VM.....	7
1.4. Your Lab VM Desktop — A Quick Tour.....	7
1.5. Open the Folder from the Command Line.....	9
1.6. A Quick Tour of the VS Code Window.....	10
1.7. Explore the Project.....	12
1.8. Install the Live Server Extension.....	13
1.9. Run the App.....	13
1.10. Initialize Git for This Folder.....	14
1.11. Make a One-Line Change.....	16
1.12. See the Change in Git.....	17
1.13. See the Same Thing from the Command Line.....	18
1.14. Clean Up.....	18
1.15. Review.....	19

Evergreen Insurance Quote Page — First Look

LAB 1

In this lab, you will open the Evergreen Insurance Quote Page in Visual Studio Code, run it in your browser, make a single one-line change, and see how VS Code surfaces that change in its Source Control panel.

You are not writing code this week. The goal is to get comfortable with the three tools your engineers use every day — the command line, Git, and VS Code — while looking at a realistic insurance web app.

Tip

The Evergreen app is shared across every week of this course. You'll come back to it in HTML week, CSS week, and beyond. For this week, just get oriented.

1.1. A Note About Keyboard Shortcuts

Your lab VM runs inside your browser. Some keyboard shortcuts (like `Ctrl+Shift+X`, `Ctrl+S`, `Ctrl+F`) may not always reach the VM — sometimes your browser catches them first. If a shortcut doesn't do anything, that's not you doing something wrong; just use the mouse/menu alternative that appears next to every shortcut in this lab. Every step in this lab lists both.

1.2. Why You're Doing This Lab

Before an engineer writes a single line of real code on any given day, they:

1. Open a project folder in their editor.
2. Run it locally to confirm it works.
3. Check Git to see what's changed since their last commit.

This is the muscle memory of professional software work. Watching your own hand do it once — even on a small change — will make every future conversation with your engineers more grounded. When they say "I ran it locally and the build was green," you'll know exactly what that looks like.

1.3. What's Already Set Up on Your Lab VM

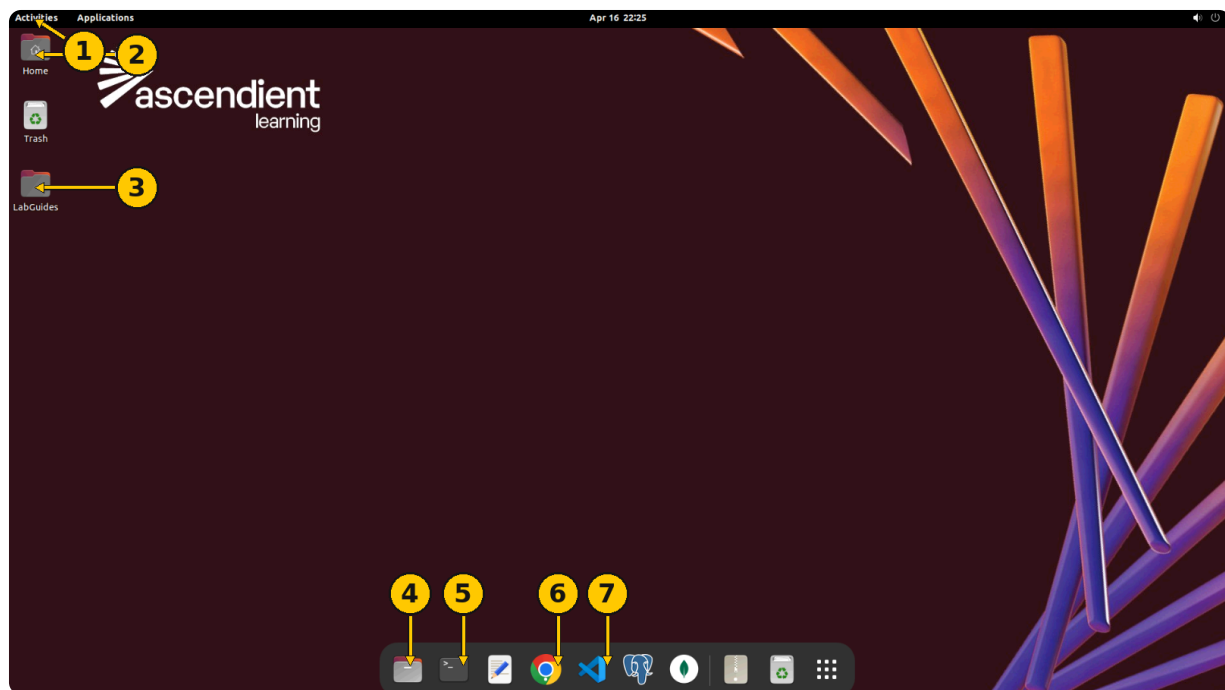
Your lab machine runs Ubuntu Linux and comes pre-installed with everything you need for this lab. You do not need to install anything.

- **Visual Studio Code** — the code editor you will use throughout the program.
- **Git** — the version control tool engineers use to track changes to code.
- **Firefox** and **Chrome** — either browser can be used to view the Evergreen quote page.
- **A terminal application** — for typing commands directly to the computer.

You will install one VS Code **extension** during this lab (Live Server), because it's used only for web-focused labs. The step-by-step below covers that.

1.4. Your Lab VM Desktop — A Quick Tour

When you connect to your lab VM, you will see a desktop that looks like the screenshot below. The numbered callouts highlight the seven items you will use most in this lab.



1. **Activities menu** (top-left corner) — click this if you need to find any app that is not on the dock. Typing a name in the search box filters the app list.
2. **Home folder** (desktop icon) — your personal folder. Inside it you will find a `Labfiles` folder that holds all the starter files for every lab in this course, including `evergreen-quote-first-look` for this lab.
3. **LabGuides folder** (desktop icon) — where the written instructions for each lab live on the VM. You can open it any time to re-read a step.
4. **Files** (dock, orange folder icon) — a graphical file browser, similar to Windows File Explorer or macOS Finder. Useful for browsing folders visually instead of from the command line.
5. **Terminal** (dock, black icon with `>_`) — the command line. You will use this to run `cd`, `ls`, `git status`, and `code .` and other commands.
6. **Chrome browser** (dock, colorful circular icon) — one of the two browsers installed on the VM. Either Chrome or Firefox can be used to view the Evergreen quote page.
7. **Visual Studio Code** (dock, blue angular icon) — the code editor you will use for the rest of this lab and every lab after it.

 Tip

If the dock is hidden, move your mouse to the bottom of the screen and it will reappear.

1.5. Open the Folder from the Command Line

You'll navigate to the lab folder using the terminal, then open it in VS Code from the command line. This is how most engineers open projects — faster than clicking through a file browser.

1. Open a terminal. Three ways — use whichever feels easiest:

- **From the dock:** Click the **Terminal** icon in the dock at the bottom of the screen (the black icon with `>_` — callout (5) on the desktop tour above).
- **From Activities:** Click **Activities** in the top-left corner, type `Terminal` in the search box, then click the Terminal icon in the results.
- **With a keyboard shortcut:** Press `Ctrl+Alt+T`.

2. Navigate into the lab folder:

```
cd ~/Labfiles/evergreen-quote-first-look
```

 Note

If you get "No such file or directory," run `ls ~/Labfiles` to see what's actually there and adjust the path. The `~` symbol is shorthand for your home folder.

3. Confirm you're in the right place:

```
ls
```

You should see `index.html`, `styles.css`, `app.js`, and `README.md`.

Why this step: `ls` is one of the commands you will see engineers use most. It answers the question "what's in this folder?" — the terminal equivalent of

opening a file browser. Every engineer starting work on a new project runs `ls` before they do anything else.

4. Open the folder in VS Code from the terminal:

```
code .
```

The `.` means "the current folder." VS Code will open with this folder loaded in its sidebar.

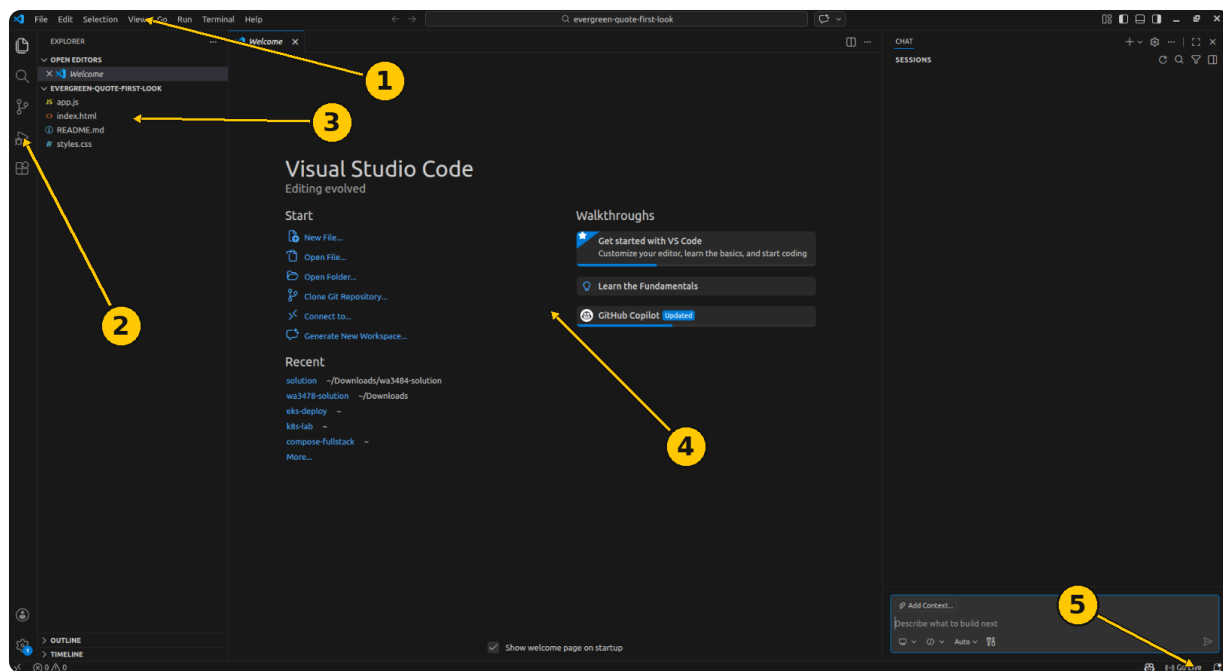
Note

The first time you open a folder, VS Code may show a dialog titled "**Do you trust the authors of the files in this folder?**" This is a safety prompt that appears on any folder VS Code has not seen before — it's not specific to this lab. The Evergreen files are provided by your course, so click **Yes, I trust the authors**. If you click **No**, VS Code opens the folder in a restricted mode that will prevent some of the steps later in this lab from working.

Why this step: Engineers almost never open VS Code from the application menu and then go File → Open Folder. They open a terminal, `cd` into the project, and type `code .`. It connects the command line to the editor in one motion and is a signal of someone who has done this a thousand times.

1.6. A Quick Tour of the VS Code Window

Now that VS Code is open, take a moment to orient yourself. The numbered callouts below highlight the five areas you will use most in this lab.



1. **Menu bar** (very top) — the familiar application menus: File, Edit, Selection, View, Go, Run, Terminal, Help. Later in this lab you will use **File > Save** to save your change and **Terminal > New Terminal** to open a terminal inside VS Code.
2. **Activity Bar** (vertical strip of icons on the far left) — the main navigation for the sidebar. Each icon switches the sidebar to a different panel: Explorer (files), Search, Source Control (Git), Run & Debug, and Extensions. You'll use several of these in this lab.
3. **Explorer panel** (sidebar, just right of the Activity Bar) — shows the project files for the folder you opened. Under **EVERGREEN-QUOTE-FIRST-LOOK** you should see four files: `app.js`, `index.html`, `README.md`, and `styles.css`. Clicking a file opens it in the editor.
4. **Editor area** (the large center region) — where files open for viewing and editing. Each open file becomes a tab at the top. The first time VS Code opens, it shows a **Welcome** tab; you can close it by clicking the `x` on the tab.
5. **Status bar / Go Live button** (the thin bar along the very bottom, right-hand side) — you will see a **Go Live** button here after you install the Live Server

extension in the next section. Clicking it starts a local web server and opens your app in the browser.

Tip

VS Code has many more features than this lab will cover. You only need the five areas above today — you'll meet more of VS Code as the weeks go on.

1.7. Explore the Project

1. Look at the **Explorer** panel on the left side of the VS Code window. Two ways to open it:
 - **With the mouse:** Click the top icon in the Activity Bar (the vertical strip of icons on the far left). It looks like two overlapping pages.
 - **With a keyboard shortcut:** Press `Ctrl+Shift+E`.
2. You should see four files listed in the Explorer:
 - `index.html` — the page structure
 - `styles.css` — the visual styling
 - `app.js` — the behavior (the premium calculation)
 - `README.md` — a short description of the app
3. Click `index.html` in the Explorer to open it in the editor.
4. Scroll through the file briefly. You do not need to understand the tags. Notice only the structure: there is a header, a hero section with a headline and a tagline, a form, and a footer. This is the HTML "skeleton" of every web page.

Why this matters: When your engineers talk about "the markup" or "the HTML," this is what they mean. You will look at it more closely in HTML week (Week 3). For now, just notice that the page is readable text, not a binary file.

1.8. Install the Live Server Extension

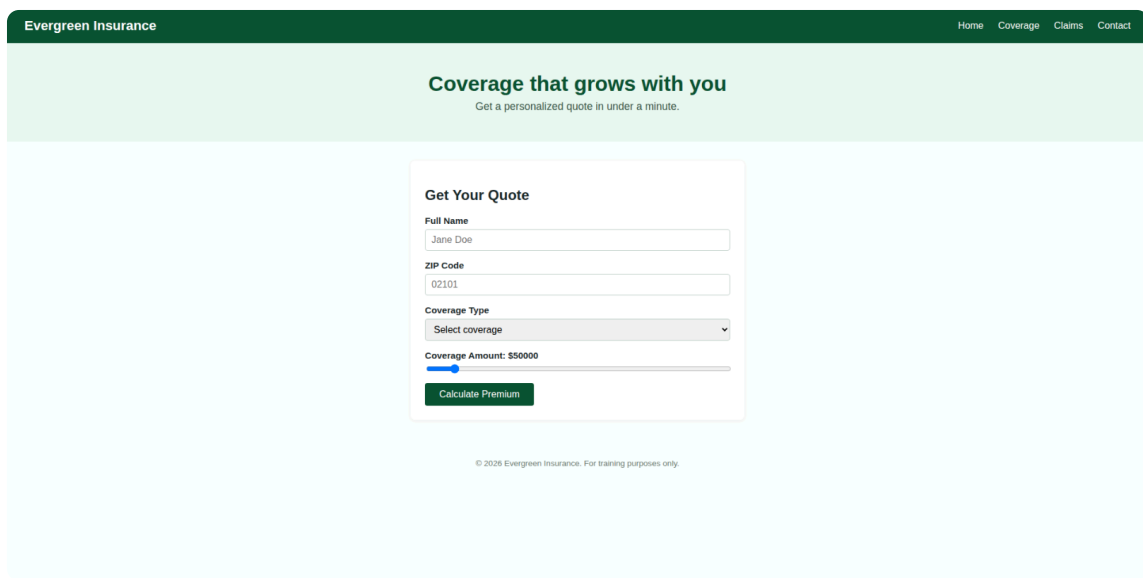
Live Server is a VS Code extension that automatically refreshes your browser every time you save a file. You will use it to run and preview the Evergreen app.

1. Open the Extensions panel. Two ways:
 - **With the mouse:** Click the fifth icon down in the Activity Bar (it looks like four small squares with one square pulled away).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+X`.
2. In the search box at the top, type `Live Server`.
3. Find the extension named **Live Server** by Ritwick Dey and click the **Install** button next to it.

Why this step: Extensions are small add-ons that give VS Code extra abilities. Most engineers have a handful of favorites. Installing one during the lab lets you see how the extensions system works — it's the same process for any extension you might install later.

1.9. Run the App

1. Go back to the Explorer panel. Two ways:
 - **With the mouse:** Click the top icon in the Activity Bar (the two overlapping pages).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+E`.
2. Right-click on `index.html` and choose **Open with Live Server**.
3. The Evergreen quote page should open in your browser (Firefox or Chrome — either works). It should look like the screenshot below.



The screenshot shows the Evergreen Insurance website. At the top is a dark green header with the logo 'Evergreen Insurance' on the left and navigation links 'Home', 'Coverage', 'Claims', and 'Contact' on the right. Below the header is a light green banner with the text 'Coverage that grows with you' and 'Get a personalized quote in under a minute.' The main content area is light blue and features a white 'Get Your Quote' form. The form has the following fields: 'Full Name' (text input with 'Jane Doe'), 'ZIP Code' (text input with '02101'), 'Coverage Type' (dropdown menu with 'Select coverage'), and 'Coverage Amount: \$50000' (slider). A green 'Calculate Premium' button is at the bottom of the form. Below the form, there is a small copyright notice: '© 2026 Evergreen Insurance. For training purposes only.'

Try it out: enter a name, a ZIP code, pick a coverage type, drag the slider, and click **Calculate Premium**. An estimated monthly premium should appear below the form.

Why Live Server: It reloads the browser every time you save a file. You will see that in the next section. Without it, you'd have to manually refresh the browser after every change, which gets tedious fast.

4. Leave the browser tab open. Switch back to VS Code.

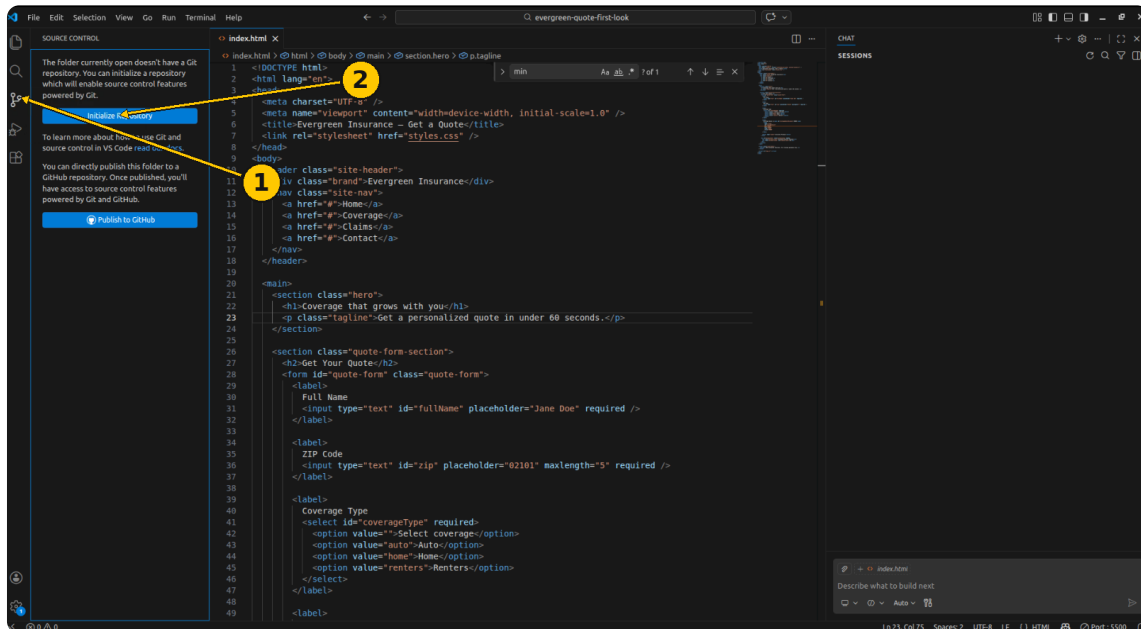
1.10. Initialize Git for This Folder

Up to now, nothing has been tracking your project in version control. Git does not automatically track every folder — you have to tell Git to start tracking this one. Engineers do this once, at the start of every new project.

1. Open the Source Control panel. Two ways:

- **With the mouse:** Click the third icon down in the Activity Bar (it looks like a branching line with dots — callout (2) from the VS Code tour above).
- **With a keyboard shortcut:** Press `Ctrl+Shift+G`.

2. You should see a panel with a message explaining the folder doesn't have a Git repository yet, along with a blue **Initialize Repository** button.



The numbered callouts point to (1) the Source Control icon in the Activity Bar that you just clicked, and (2) the blue **Initialize Repository** button you will click in the next step.

3. Click **Initialize Repository**. VS Code creates a hidden `.git` folder inside your project. All four of your project files will now appear in the Source Control panel under **Changes** with a small `U` next to each name. `U` stands for "untracked" — Git has seen the files but they aren't part of any commit yet.

Why this step: Every real engineering project starts the same way: initialize Git, commit the starting files once as a baseline, and from then on every future change shows up as a modification against that baseline. Without this step, Git has nothing to compare your changes against, and VS Code's Source Control panel can't tell you what changed.

4. Make an initial commit so there is a baseline to compare future changes against:
 - Click inside the **Message** box at the top of the Source Control panel (it shows grey placeholder text like "Message (press Ctrl+Enter to commit)").
 - Type `Initial commit`.
 - Click the blue **Commit** button just below the message box. If VS Code asks "Would you like to stage all your changes and commit them directly?", click **Yes**.
5. The Source Control panel should now be empty under **Changes** — everything has been committed. Your project has exactly one commit in its history. From this point on, any edit you make to any file will show up in the Source Control panel as a modification.

1.11. Make a One-Line Change

You are going to change the page's tagline from `Get a personalized quote in under a minute.` to `Get a personalized quote in under 60 seconds.`

1. Make sure `index.html` is the file currently open in the editor. If not, click it in the Explorer.
2. Find the tagline line. Two ways:
 - **With the mouse:** Use the scroll bar on the right side of the editor and scroll until you see a line that contains `under a minute`.
 - **With a keyboard shortcut:** Press `Ctrl+F` to open the Find box, type `under a minute`, and press `Enter` — the editor will jump to that line.
3. Change the line so it reads:

```
<p class="tagline">Get a personalized quote in under 60  
seconds.</p>
```

4. Save the file. Two ways:
 - **With the mouse:** Click the **File** menu at the top of the VS Code window, then click **Save**.

- **With a keyboard shortcut:** Press `Ctrl+S`.
5. Switch to your browser tab. The page should update automatically — the tagline now reads the new text.

Why this matters: The "edit file, save, see it update in the browser" cycle is the inner loop of front-end development. Your engineers do this hundreds of times a day. When they talk about "hot reload" or "fast feedback," this is what they mean.

1.12. See the Change in Git

VS Code has a built-in view for Git. Even though you have not run any `git` commands yourself, VS Code has been tracking your change.

1. Open the Source Control panel. Two ways:
 - **With the mouse:** Click the third icon down in the Activity Bar (it looks like a branching line with dots).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+G`.
2. You should see `index.html` listed with an `M` next to it. `M` means "modified."

Why this matters: This view shows the same information you would get from running `git status` in the terminal. Engineers use this view constantly to see exactly what they have changed before they commit. When an engineer says "let me just sanity-check my changes," this is the panel they are looking at.

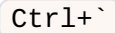
3. Click on `index.html` in the Source Control panel. VS Code will open a **diff view** showing the old line on the left (highlighted red) and your new line on the right (highlighted green).

Why this matters: The diff view is the fundamental unit of code review. When an engineer opens a pull request, this is exactly what their teammates see. You will revisit this in Week 2 (GitHub). For now, just notice how focused it is: only what changed, nothing else.

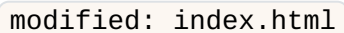
4. Close the diff view by clicking the  on its tab at the top of the editor.

1.13. See the Same Thing from the Command Line

You have just seen Git through VS Code's graphical panel. Now see it through the terminal, so you know the two views show the same thing.

1. Open VS Code's integrated terminal. Two ways:
 - **With the mouse:** Click the **Terminal** menu at the top of the VS Code window, then click **New Terminal**.
 - **With a keyboard shortcut:** Press .
2. The terminal opens inside your project folder. Run:

```
git status
```

You should see  — the same information the Source Control panel showed you.

3. Now run:

```
git diff
```

You'll see the same before/after change — this time in text form, with red  and green  marks.

Why this matters: The graphical and command-line views of Git are just two windows into the same thing. Some engineers prefer the terminal, others prefer the visual panel, and most switch between them. You don't have to pick a favorite. You just have to know they're equivalent.

1.14. Clean Up

The lab environment will be reset between sessions, but practice cleanup anyway.

1. In the terminal, discard your change so the folder is back to its starting state:

```
git checkout index.html
```

Why this step: `git checkout <file>` throws away uncommitted changes to that file. Engineers use this when they realize they went down the wrong path and want a clean slate. Important: this **deletes your work** — use it only when you are sure.

2. Confirm the change is gone:

```
git status
```

It should say `nothing to commit, working tree clean`.

1.15. Review

In this lab, you:

- Navigated to a project folder from the command line using `cd` and `ls`.
- Opened a folder in VS Code from the terminal using `code .`.
- Installed a VS Code extension (Live Server).
- Ran a small web app in your browser.
- Edited a line of HTML and watched the browser hot-reload.
- Viewed the change in VS Code's Source Control panel.
- Viewed the same change from the command line with `git status` and `git diff`.
- Discarded the change with `git checkout`.

You have not written any code. You have watched the surface — the tools and the feedback loops — of what your engineers do every day. That context is what the rest of this program builds on.